PawAdopt — VS Code Setup & Render Publishing Guide

Step-by-step: editor setup, API keys, GitHub push, and Render deployment of the PHP build

October 27, 2026

Table of Contents

PawAdopt — VS Code Setup & Render Publishing Guide

Before you start: Visual Studio is the wrong editor for this project

PawAdopt is a PHP 8 + MySQL web application — the repository contains no .NET project (`.csproj`), no Node `package.json`, and no Python `requirements.txt`. Microsoft **Visual Studio** is built for .NET/C++ projects and would fight you here. The right tool is **Visual Studio Code** (the free, lightweight editor from the same company — different product). Every step below uses **VS Code**. If you specifically wanted Visual Studio for another project of yours, the concepts transfer, but PawAdopt itself should be edited in VS Code.

What the API keys actually are (verified in the code): PawAdopt reads exactly one third-party API key — `GEMINI_API_KEY` (Google AI Studio), used only server-side in `login.php` for the PawBot assistant. Everything else is either a **database credential** (`DB_*` from your MySQL host) or an **e-mail credential** (`SMTP_*` — a Gmail app-password). There are **no Stripe, OpenAI, or Supabase keys** — the code never references them. Render automatically injects `PORT` — you never set it yourself.

Part 1 — Install the tools (one-time, ~20 minutes)

1. **Visual Studio Code** — download from https://code.visualstudio.com and install (accept the default "Add to PATH" option).

2. **Git for Windows** — https://git-scm.com/download/win → run the installer → accept the defaults (this gives you `git` and Git Bash).

3. **PHP 8.2 locally** — install **XAMPP** from https://www.apachefriends.org (one installer = Apache + PHP 8.2 + MySQL/MariaDB + phpMyAdmin). You need PHP even for editing, because VS Code's PHP extension uses the PHP binary for linting.

4. **VS Code PHP extension** — open VS Code → Extensions (`Ctrl+Shift+X`) → search **"PHP Intelephense"** (by Ben Mewburn) → Install. Then `Ctrl+,` (Settings) → search "php.validate.executablePath" → **Edit in settings.json** → add:

```
"php.validate.executablePath": "C:\\xampp\\php\\php.exe"
```

(adjust if you installed XAMPP elsewhere). This gives you live error checking in every PHP file.

Part 2 — Open the project and configure it locally

1. **Get the code.** Download `PawAdopt_Render_Ready_1027.zip` (link in the chat, verified size 44,170,004 bytes, SHA-256 `B83258A2CCC00FCF83E8E4FFC8657A40A4501452031017E73C5A67437FE39066` — see the Windows Download Guide for the verification steps), extract it to `C:\Projects\PawAdopt\`.

2. **Open in VS Code:** File → Open Folder → `C:\Projects\PawAdopt`.

3. **Create your local `.env`** (the project loads it via `config/env.php`; real environment variables always win). In VS Code: right-click in the Explorer → New File → name it `.env` → paste, then edit the values:

   ```
   DB_HOST=127.0.0.1
   DB_PORT=3306
   DB_NAME=pawadopt
   DB_USER=root
   DB_PASS=
   APP_URL=http://localhost:8080/PawAdopt
   APP_ENV=development
   ```

   (XAMPP MySQL defaults: user `root`, empty password, port 3306.)

4. **Create the local database:** start the XAMPP Control Panel → **Start** Apache and MySQL → open http://localhost/phpmyadmin → New → database name `pawadopt` → Create → Import tab → import the SQL files **in this order**: `sql/pawadopt.sql` → `sql/appointments.sql` → `sql/feedback_reports.sql` → `sql/password_reset_otps.sql` → `sql/account_deletion_requests.sql` → `sql/user_prefs.sql` → `sql/security_hardening.sql` → `sql/ui_customization.sql`.

5. **Run the app locally.** In VS Code: Terminal → New Terminal (`Ctrl+` `) → run: ```bash php -S localhost:8080 ``` Wait — for the folder layout to matchAPP_URL`, serve the *parent* directory:

   ```
   cd C:\Projects
   php -S localhost:8080
   ```

   Then open **http://localhost:8080/PawAdopt/index.php** in your browser. The landing page should load. Register a test account to confirm the database connection works.

6. **Never commit `.env`.** In the VS Code terminal: `printf ".env\n" > .gitignore`.

Local Gemini/SMTP keys are optional — the app runs fine without them; only the PawBot chat and password-reset e-mail features stay disabled until you add `GEMINI_API_KEY` / `SMTP_PASSWORD` to the `.env`.

Part 3 — Push the project to GitHub

1. Create an empty repository on https://github.com/new — name: `pawadopt`, **Private** is fine, do **not** initialize with a README (the project has one).

2. In the VS Code terminal (inside `C:\Projects\PawAdopt`):

   ```
   git init
   git add .
   git commit -m "PawAdopt secured build (Render-ready)"
   git branch -M main
   git remote add origin https://github.com/<your-
   username>/pawadopt.git
   git push -u origin main
   ```

   When the browser pops up, sign in to GitHub and authorize Git Credential Manager.

3. Verify on the GitHub repo page that **no `.env` file appears** — only `.env.example`.

Part 4 — Create the production MySQL database (free)

Render has **no managed MySQL** (its managed database is PostgreSQL only), and PawAdopt needs MySQL/MariaDB. Use **Aiven's free MySQL tier** (or any MySQL host):

1. Sign up at https://aiven.io → **Create service** → **MySQL** → free plan → choose a region near your Render region.
2. When running, open **Overview** → **Connection information** and copy: **Host** (e.g. `mydb-mysql-x.a.aivencloud.com`), **Port** (Aiven free uses a custom port — copy *theirs*), **User** (`avnadmin`), **Password**, **Database** (`defaultdb`).
3. Import the same 8 SQL files from Part 2 step 4 into this new database (same order, same client — you can use Aiven's web console or MySQL Workbench).
4. These six values become the Render `DB_*` variables in Part 5.

Part 5 — Publish to Render

Option A — Blueprint (recommended; uses the `render.yaml` in the repo)

1. Sign up / log in at https://dashboard.render.com
2. **New +** → **Blueprint** → connect your GitHub account if asked → select the `pawadopt` repo → Render reads `render.yaml` and shows the `pawadopt-web` service (Docker runtime, `healthCheckPath: /health.php`, plan Free).
3. During the first sync, Render **prompts you for every `sync: false` variable** — paste the values from the table below.
4. Click **Apply**. Render builds the Docker image from `Dockerfile`.

Option B — Manual web service

1. **New +** → **Web Service** → **Git Provider** → pick the `pawadopt` repo.
2. Fill the form: **Name** `pawadopt-web` · **Region** near your database · **Branch** `main` · **Runtime/Language: Docker** (Render has no native PHP runtime — the Dockerfile is the deployment) · **Plan: Free**.
3. Under **Advanced** → **Add Environment Variable**, add everything from the table below.
4. (Optional but recommended) **Advanced** → **Health Check Path**: `/health.php`.
5. **Create Web Service** → first build starts.

Environment variables (every name verified in the project code)

Set in Render → `pawadopt-web` → **Environment** tab (values live only in the dashboard, never in Git):

Variable	Required?	Where you get the value	Notes
`DB_HOST`	**Yes**	Aiven → Connection information → Host	App refuses to start without it (fail-fast in `config/database.php`)
`DB_PORT`	**Yes**	Aiven → Connection information → Port	Aiven free ≠ 3306 — copy their port exactly
`DB_NAME`	**Yes**	Aiven → `defaultdb` (or your DB name)	
`DB_USER`	**Yes**	Aiven → `avnadmin` (or a least-privilege user you create)	
`DB_PASS`	**Yes**	Aiven → the service password	Secret — dashboard only
`APP_URL`	No	Render fills it via `RENDER_EXTERNAL_URL` in `render.yaml` (manual setup: leave unset — the app derives it from the request host)	Set only for custom domains
`APP_ENV`	No	Type `production`	Default is already `production`
`UPLOAD_PATH`	No	Only needed with a paid persistent disk: `/var/www/html/uploads`	Free tier has an ephemeral filesystem — uploaded pet photos are wiped on

Variable	Required?	Where you get the value	Notes
			redeploys
`GEMINI_API_KEY`	For PawBot chat	**Google AI Studio** → https://aistudio.google.com → Get API key → Create API key	**The one true API key.** Server-side only (read in `login.php`); unset = chat endpoint returns HTTP 500, everything else works. Rotate the key that was exposed in the old build!
`SMTP_HOST`	For e-mail	Type `smtp.gmail.com`	Default already in `config/smtp.php`
`SMTP_PORT`	For e-mail	Type `587`	
`SMTP_ENCRYPTION`	For e-mail	Type `tls`	
`SMTP_USERNAME`	For e-mail	Your Gmail address	
`SMTP_PASSWORD`	For e-mail	**Google Account** → **Security** → **2-Step Verification** → **App passwords** → generate a 16-character app-password (spaces removed)	Not your Gmail login password! Rotate the one exposed in the old build
`SMTP_FROM_EMAIL`	For e-mail	Same Gmail address	
`SMTP_FROM_NAME`	For e-mail	Type `PawAdopt`	
`PORT`	**Never set it**	Render injects it automatically	`docker/start.sh` rewrites Apache's `Listen` to `$PORT` on every boot

Part 6 — Deploy and verify

1. Watch **pawadopt-web** → **Events** until the deploy is live. The deploy only goes live if `GET /health.php` returns HTTP 200 — and `health.php` returns 200 only when the database accepts `SELECT 1`. A wrong database config therefore fails the deploy visibly instead of producing a silently broken site.
2. Open your service URL (e.g. `https://pawadopt-web.onrender.com`) — the landing page should load.
3. Quick functional checks: register a test adopter and shelter → log in to both portals → shelter adds a pet → adopter favorites + applies (submit button disables and shows

"Submitting…") → messages send both ways → `GET /api/audit_logs.php` while logged out returns HTTP 403.

4. **Free-tier realities:** the service sleeps after ~15 minutes idle (first request after that takes ~30–60 s to wake); 750 free instance-hours/month; filesystem is ephemeral (see `UPLOAD_PATH` row above).

Part 7 — Summary: exactly which keys/secrets you must provide

Secret	Is it an "API key"?	Where to obtain	Where to paste
`GEMINI_API_KEY`	**Yes — the only third-party API key in this app**	Google AI Studio (aistudio.google.com → Get API key)	Render env var (and local `.env` if you want chat locally)
`DB_HOST` / `DB_PORT` / `DB_NAME` / `DB_USER` / `DB_PASS`	No — database credentials	Aiven (or any MySQL host) connection panel	Render env vars
`SMTP_PASSWORD` (+ `SMTP_USERNAME`, `SMTP_FROM_EMAIL`)	No — Gmail **app-password**	Google Account → Security → App passwords	Render env vars
`PORT`	No	Render injects it	Nothing to do

Before going live: revoke and regenerate the Gemini key and Gmail app-password that were exposed in the original build (they were found hardcoded in `login.php` and `config/smtp.php` during the audit) — the new values are what you paste into Render.

Troubleshooting quick list

- **Deploy fails at build** → check Events for the failing Dockerfile step; confirm the repo has no real `.env`.
- **502 / "no open ports"** → the boot log must show `PawAdopt: Apache listening on 0.0.0.0:<port>`; if not, `docker/start.sh` isn't running (entrypoint changed).
- **Health check fails / "Database connection refused"** → #1 cause: wrong `DB_PORT` (Aiven ≠ 3306). Then: missing `DB_*` values, wrong password, or provider IP allow-listing.
- **Photos disappear after redeploy** → free-tier ephemeral filesystem; attach a paid persistent disk or accept re-uploads.
- **Users logged out after each deploy** → same ephemeral cause (file sessions).
- **OTP e-mail missing** → `SMTP_PASSWORD` must be a Gmail **app-password**, with 2-Step Verification enabled; ~100–150 mails/day limit.

Full deployment detail lives in `DEPLOY.md` inside the project; full download/verify steps in the Windows Download Guide.